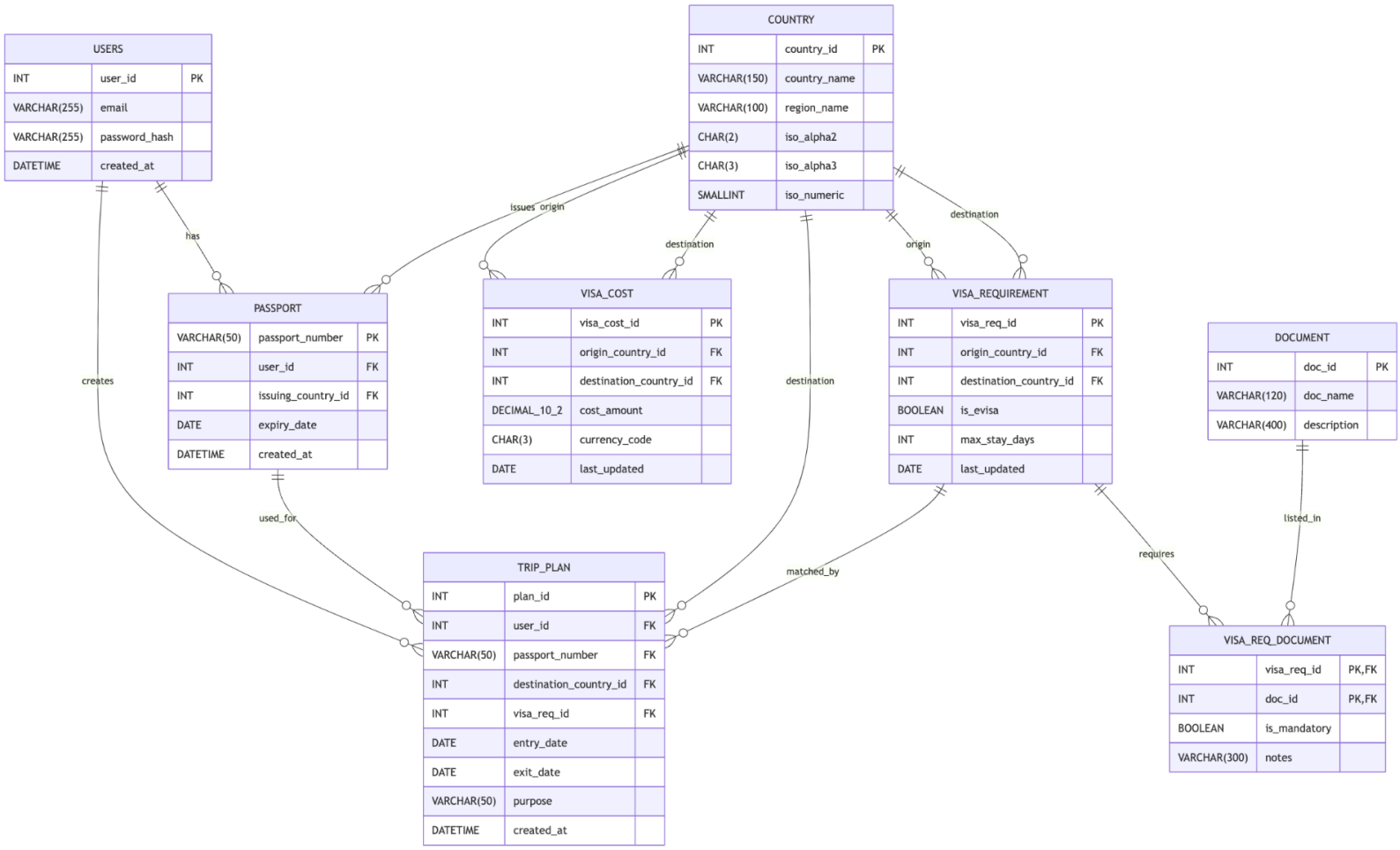
USERS
INT user_id PK
VARCHAR(255) email
VARCHAR(255) password_hash
DATETIME created_at
COUNTRY
INT country_id PK
VARCHAR(150) country_name
VARCHAR(100) region_name
CHAR(2) iso_alpha2
CHAR(3) iso_alpha3
SMALLINT iso_numeric
PASSPORT
VARCHAR(50) passport_number PK
INT user_id FK
INT issuing_country_id FK
DATE expiry_date
DATETIME created_at
VISA_COST
INT visa_cost_id PK
INT origin_country_id FK
INT destination_country_id FK
DECIMAL_10_2 cost_amount
CHAR(3) currency_code
DATE last_updated
VISA_REQUIREMENT
INT visa_req_id PK
INT origin_country_id FK
INT destination_country_id FK
BOOLEAN is_evisa
INT max_stay_days
DATE last_updated
DOCUMENT
INT doc_id PK
VARCHAR(120) doc_name
VARCHAR(400) description
TRIP_PLAN
INT plan_id PK
INT user_id FK
VARCHAR(50) passport_number FK
INT destination_country_id FK
INT visa_req_id FK
DATE entry_date
DATE exit_date
VARCHAR(50) purpose
DATETIME created_at
VISA_REQ_DOCUMENT
INT visa_req_id PK,FK
INT doc_id PK,FK
BOOLEAN is_mandatory
VARCHAR(300) notes
has
issues
origin
destination
destination
origin
destination
creates
used_for
matched_by
requires
listed_in

Entities

1. USERS
    a. Represents an account for data storage and authentication.
    b. Attributes:
        i. user_id (Primary Key): unique key assigned to each account.
        ii. email: used as login identifier.
        iii. password_hash: store hashed password.
        iv. created_at
    c. Entity, not an attribute, because User is associated with one or more Trip, another entity, and it needs to be created, updated, and deleted independently.
2. COUNTRY
    a. List of countries. Used everywhere in the app as passport issuers, travel destinations etc.
    b. Attributes
        i. country_id (Primary Key)
        ii. country_name: display name (canonical).
        iii. region_name: geographic location of the country, used for filtering. Each country belongs to exactly one region.
        iv. iso_alpha2, iso_alpha3, iso_numeric: standardized codes for joining across datasets.
    c. Entity, not an attribute, because it is referenced multiple times in the application and used in different scenarios.
3. PASSPORT
    a. Represents a passport owned by a specific user. For a specific country.
    b. Attributes
        i. passport_number (Primary Key)
        ii. user_id (FK → USERS): owner of the passport.
        iii. issuing_country_id (FK → COUNTRY): country that issued this passport.
        iv. Expiry_date
        v. created_at
    c. Entity, not an attribute, because a user can have multiple passports, and this support this one-to-many relationship.
4. VISA_REQUIREMENT
    a. Represents the visa rule for a person owning a passport from the origin country to travel to the destination country.
    b. Attributes
        i. visa_req_id (Primary Key)
        ii. origin_country_id (FK → COUNTRY): the passport issuing country.
        iii. destination_country_id (FK → COUNTRY): the target country.
        iv. is_evisa (BOOLEAN): whether e-visa is available/required.
        v. max_stay_days (INT): maximum allowed stay.
        vi. last_updated: the last time we updated this visa rule.

    c. Entity, not an attribute, because visa rules are different for different two countries, so it can't be an attribute.
5. VISA_COSTS
    a. The cost of obtaining a visa to the destination country for a person from origin country of origin.
    b. Attributes
        i. visa_cost_id (PK)
        ii. origin_country_id (FK → COUNTRY): country that the user has a passport from
        iii. destination_country_id (FK → COUNTRY): the country the user wants to go to
        iv. cost_amount
        v. currency_code
        vi. last_updated: the last time we updated this visa rule.
6. TRIP_PLAN
    a. Represents a saved trip plan created by a particular user.
    b. Attributes
        i. plan_id (Primary Key)
        ii. user_id (FK → USERS): creator of the plan.
        iii. destination_country_id (FK → COUNTRY): where the user is going.
        iv. passport_number (FK→PASSPORT): the passport that we use to search for the visa requirement.
        v. visa_req_id (FK → VISA_REQUIREMENT): the matching visa rule for destination country and the passport country (select the best one)
        vi. entry_date: date of entry of the destination country.
        vii. exit_date: date of exit of the destination country. Combined with entry_date, used to compute trip length and compare against max_stay_days.
        viii. Purpose: the purpose of the trip (e.g. tourist, student, etc.)
        ix. Created_at.
    c. Entity, not an attribute, because one user can plan multiple trips, so it is one-to-many and can't be stored as attributes in Users.
7. DOCUMENT
    a. Represents catalog of document types that may be required
    b. Attributes
        i. doc_id (PK)
        ii. doc_name: name of the document
        iii. Description: additional info we might want to show users.
    c. A visa requirement can require multiple documents, and the same document can be required by many different visa requirements.
8. VISA_REQ_DOCUMENT
    a. Represents a document needed to apply for a specific visa.
    b. Attributes
        i. visa_req_id (PK, FK → VISA_REQUIREMENT)

        ii. doc_id (PK, FK → DOCUMENT)
        iii. Is_mandatory: whether this document is mandatory for a visa to pass.
        iv. Notes: potential addition notes
    c. Entity, not an attribute, because the relationship between VISA_REQUIREMENT and DOCUMENT has its own attributes.

Relationships

1. USERS—has→ PASSPORT
    a. One user can have 1..many passports (many for dual citizenship etc.)
    b. Each passport belongs to exactly 1 user.
2. PASSPORT —issues→ COUNTRY
    a. Each passport can be issued by exactly 1 country.
    b. One country can issue 0..many passports.
    c. Passport's issuing_country_id must match the origin_country_id in the visa requirement referenced by a trip.
3. USERS —creates→TRIP_PLAN
    a. One user can have 0..many plans
    b. Each trip_plan is made by exactly 1 user
4. TRIP_PLAN —go to→COUNTRY
    a. Each plan goes to exactly 1 country
    b. Each country can be the destination for 0..many trip plans
5. COUNTRY—origin/destination→ VISA_REQUIREMENT
    a. Each country appears in 0..many visa requirements
    b. Each visa requirement mentions exactly 1 origin and 1 destination
6. COUNTRY—origin/destination→ VISA_COST
    a. Each country appears in 0..many visa costs
    b. Each visa cost mentions exactly 1 origin and 1 destination
7. TRIP_PLAN —matched_by→ VISA_REQUIREMENT
    a. Each trip plan reference exactly 1 visa_requirement
    b. Each VISA_REQUIREMENT can be referenced by 0..many TRIP_PLANs.
8. VISA_REQUIREMENT—requires→ DOCUMENT
    a. Each visa application requires 0..many documents.
    b. Each document is required in 0..many visa applications.

Database Normalization (We are applying 3NF)

To summarize the above, here is the universal relation set R in our database:

R(
user_id, email, password_hash,
country_id, country_name, region_name, iso_alpha2, iso_alpha3, iso_numeric,
passport_number, expiry_date, issuing_country_id,
origin_country_id, destination_country_id, is_evisa, max_stay_days,
visa_req_id, purpose, visa_cost_id, cost_amount, currency_code,

plan_id, entry_date, exit_date, created_at, last_updated, doc_id, doc_name, description, is_mandatory, notes
)

Here is the functional dependencies we have:
Users:
user_id -> email, password_hash, created_at
email -> user_id

Country:
country_id -> country_name, region_name, iso_alpha2, iso_alpha3, iso_numeric

Passport:
passport_number -> user_id, issuing_country_id, expiry_date, created_at

Visa Requirement:
visa_req_id -> origin_country_id, destination_country_id, is_evisa, max_stay_days, last_updated
origin_country_id, destination_country_id -> is_evisa, max_stay_days
origin_country_id, destination_country_id -> visa_req_id
Visa Costs:
visa_cost_id -> origin_country_id, destination_country_id, cost_amount, currency_code, last_updated
origin_country_id, destination_country_id -> cost_amount, currency_code
origin_country_id, destination_country_id -> visa_cost_id

Trip Plan:
plan_id → user_id, destination_country_id, visa_req_id, entry_date, exit_date, created_at, passport_number, purpose

Computing minimal basis for FDs

We separate FDs to basis such as:
user_id -> email, password_hash
to
user_id -> email
user_id ->password_hash

We merge the FDs with the same LFH such as:
origin_country_id, destination_country_id -> cost_amount, currency_code
origin_country_id, destination_country_id -> visa_cost_id
to
origin_country_id, destination_country_id -> cost_amount, currency_code, visa_cost_id

(same for the visa requirements)

We remove redundant FDs such as:
remove email -> password_hash because email -> user_id and user_id -> password_hash

After all we have this minimal basis G:
USERS
user_id → email
user_id → password_hash
user_id → created_at
email → user_id

PASSPORT
passport_number → user_id
passport_number → issuing_country_id
passport_number → expiry_date
passport_number → created_at

COUNTRY
country_id → country_name
country_id → iso_alpha2
country_id → iso_alpha3
country_id → iso_numeric
country_id → region_name

VISA_REQUIREMENT
visa_req_id → origin_country_id
visa_req_id → destination_country_id
visa_req_id → is_evisa
visa_req_id → max_stay_days
origin_country_id, destination_country_id → visa_req_id

VISA_COSTS
visa_cost_id → origin_country_id
visa_cost_id → destination_country_id
visa_cost_id → cost_amount
visa_cost_id → currency_code
origin_country_id, destination_country_id → visa_cost_id
origin_country_id, destination_country_id → cost_amount
origin_country_id, destination_country_id → currency_code

TRIP_PLAN
plan_id → user_id
plan_id → destination_country_id

plan_id → visa_req_id
plan_id → entry_date
plan_id → exit_date
plan_id → created_at
plan_id → purpose
plan_id → passport_number

DOCUMENT
doc_id → doc_name
doc_id → description

VISA_REQ_DOCUMENT
(visa_req_id, doc_id) → is_mandatory
(visa_req_id, doc_id) → notes

Decomposing to 3NF, we finally have the normalized database:
USERS(user_id, email, password_hash, created_at)
COUNTRY(country_id, country_name, region_name, iso_alpha2, iso_alpha3, iso_numeric)
PASSPORT(passport_number, user_id, issuing_country_id, expiry_date, created_at)
VISA_REQUIREMENT(visa_req_id, origin_country_id, destination_country_id, is_evisa, max_stay_days, last_updated)
VISA_COST(visa_cost_id, origin_country_id, destination_country_id, cost_amount, currency_code, last_updated)

TRIP_PLAN(plan_id, user_id, passport_number, destination_country_id, visa_req_id, entry_date, exit_date, purpose, created_at)

DOCUMENT(doc_id, doc_name, description)

VISA_REQ_DOCUMENT(visa_req_id, doc_id, is_mandatory, notes)

Relational Schema (Logical Design):

```
USERS(
   user_id INT [PK],
   email VARCHAR(255) UNIQUE,
   password_hash VARCHAR(255),
   created_at DATETIME
)


COUNTRY(
```

```
   country_id INT [PK],
   country_name VARCHAR(150),
   region_name VARCHAR(100),
   iso_alpha2 CHAR(2),
   iso_alpha3 CHAR(3),
   iso_numeric SMALLINT
)

PASSPORT(
   passport_number VARCHAR(50) [PK],
   user_id INT [FK to USERS(user_id)],
   issuing_country_id INT [FK to COUNTRY(country_id)],
   expiry_date DATE,
   created_at DATETIME
  )


VISA_REQUIREMENT(
         visa_req_id INT [PK],
         origin_country_id INT [FK to COUNTRY(country_id)],
         destination_country_id INT [FK to COUNTRY(country_id)],
         is_evisa BOOLEAN,
         max_stay_days INT,
         last_updated DATE,
         UNIQUE (origin_country_id, destination_country_id)
)

VISA_COST(
   visa_cost_id INT [PK],
   origin_country_id INT [FK to COUNTRY(country_id)],
   destination_country_id INT [FK to COUNTRY(country_id)],
   cost_amount DECIMAL(10,2),
   currency_code CHAR(3),
   last_updated DATE,
   UNIQUE(origin_country_id, destination_country_id)
)

TRIP_PLAN(
   plan_id INT [PK],
   user_id INT [FK to USERS(user_id)],
   passport_number VARCHAR(50) [FK to PASSPORT(passport_number)],
   destination_country_id INT [FK to COUNTRY(country_id)],
   visa_req_id INT [FK to VISA_REQUIREMENT(visa_req_id)],
   entry_date DATE,
```

```
    exit_date DATE,
    purpose VARCHAR(50),
    created_at DATETIME
)

VISA_REQ_DOCUMENT(
    visa_req_id INT [FK to VISA_REQUIREMENT(visa_req_id)],
    doc_id INT [FK to DOCUMENT(doc_id)],
    is_mandatory BOOLEAN,
    notes VARCHAR(300),
    PRIMARY KEY (visa_req_id, doc_id)
)

DOCUMENT(
        doc_id INT [PK],
        doc_name VARCHAR(120),
        description VARCHAR(400)
)
```